

Module 6 - Code Quality and SonarQube

What is SonarQube?

Imagine you wrote a Java program.

```
public class Demo {  
    public static void main(String[] args) {  
        int a = 10;  
        int b = 20;  
        if(a==10){  
            System.out.println("Hello");  
        }  
    }  
}
```

The program runs perfectly.

But...

- Maybe forgot comments.
- Maybe variable names are bad.
- Maybe there is duplicate code.
- Maybe there is a security issue.
- Maybe your code is too complex.
- Maybe a future developer will struggle to understand it.

Your compiler **doesn't complain** because the code is syntactically correct.

This is where **SonarQube** comes in.

SonarQube is an open-source **Static Code Analysis Tool** that analyzes your source code and reports:

- Bugs
- Code Smells
- Security Vulnerabilities
- Duplicate Code
- Code Coverage
- Complexity
- Maintainability

Think of it as a **grammar checker for code**. Just as Grammarly finds issues in writing, SonarQube finds issues in software.

Why Do Companies Use SonarQube?

Imagine two developers.

Developer A

```
int a;
```

```
int b;
```

```
int c;
```

Developer B

```
int employeeSalary;
```

```
int employeeAge;
```

```
int employeeExperience;
```

Both programs work.

Which one is easier to understand?

Obviously, Developer B's.

SonarQube encourages writing code that is easier to read, maintain, and improve.

Real Company Workflow

Developer – GitHub - Jenkins / GitHub Actions – SonarQube - Quality Report - Deploy Application

Before the application is deployed, SonarQube checks its quality. If it doesn't meet the required standards, the deployment can be blocked.

Static Code Analysis

Static Analysis

Code is analyzed **without executing it**.

Example:

```
public class Demo{  
    int x;  
}
```

SonarQube simply reads the source code and checks for issues. The program never runs.

Runtime Testing

Here, the application actually executes.

Example:

```
Scanner sc = new Scanner(System.in);  
int a = sc.nextInt();
```

The code runs, takes input, and produces output.

Examples include:

- JUnit
- Selenium
- Manual Testing

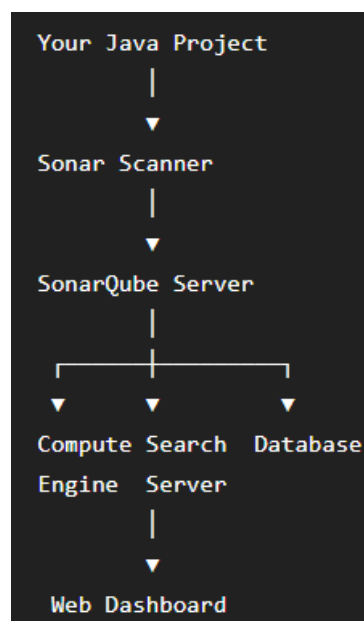
Difference

Static Analysis	Runtime Testing
Code is not executed	Code is executed
Finds coding issues	Finds runtime bugs
Faster	Slower
Uses source code	Uses running application
Example: SonarQube	Example: JUnit

Benefits of SonarQube

- Finds bugs before deployment
- Improves readability
- Detects duplicate code
- Measures complexity
- Improves security
- Increases maintainability
- Integrates with CI/CD
- Supports many programming languages
- Generates detailed reports

SonarQube Architecture



1. Sonar Scanner (Client)

This is the component that starts everything.

Think of it as a **delivery person**.

Its job is to:

- Read your source code

- Read pom.xml
- Read configuration
- Send everything to the SonarQube Server

Example command:

mvn sonar:sonar

or

mvn clean verify sonar:sonar

After this command, the scanner sends all project information to the server.

Example

Suppose your project contains:

StudentManagement

– Student.java

– Department.java

– pom.xml

The Sonar Scanner collects these files and uploads them to the SonarQube Server.

2. SonarQube Server

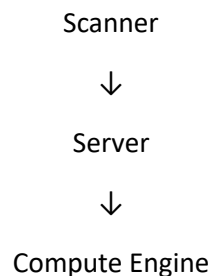
This is the **brain** of SonarQube.

Everything arrives here first.

It receives:

- Java files
- XML files
- Configuration
- Test reports
- Coverage reports

Then it forwards the analysis work to the Compute Engine.



3. Compute Engine

This is where the real analysis happens.

The Compute Engine checks:

- Bugs

- Code Smells
- Security Issues
- Duplicate Code
- Complexity
- Coverage

The Compute Engine creates a complete report.

4. Search Server (Elasticsearch)

Imagine your project has

50,000 Java files.

You search for

Student.java

Should SonarQube read all files every time?

No.

Instead, it uses **Elasticsearch**.

Its job:

- Fast searching
- Fast filtering
- Fast dashboard loading

Think of it like Google's search engine, but only for your SonarQube data.

5. Sonar Database

Where is everything stored?

In the database.

It stores:

- Project names
- Previous analyses
- Bugs
- Code Smells
- Users
- Quality Profiles
- Quality Gates
- Metrics

Example:

Project:

StudentManagement

Analysis 1

Bugs : 5

Smells : 30

Coverage : 60%

Tomorrow you scan again.

Analysis 2

Bugs : 2

Smells : 10

Coverage : 90%

All of this history is stored in the database.

6. Web Server

This is the website you open.

Usually:

http://localhost:9000

The Web Server displays:

- Dashboard
- Graphs
- Reports
- Bugs
- Coverage
- Quality Gate
- Projects

Example Dashboard:

Project: StudentManagement

Bugs: 2

Code Smells: 15

Coverage: 87%

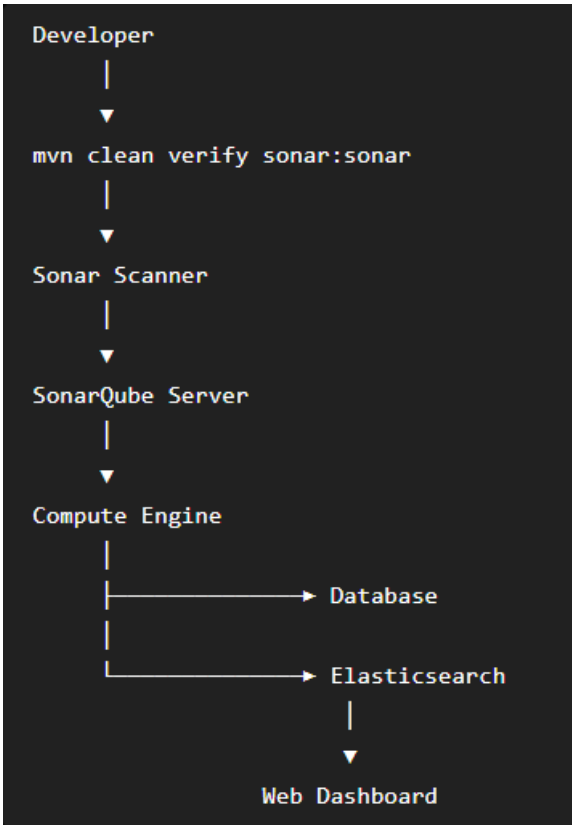
Duplications: 1.3%

Quality Gate: PASSED

You never interact directly with the Compute Engine or Database.

You only use the Web Server.

Complete Workflow



What Each Component Does

Component	Responsibility
Sonar Scanner	Collects project files and sends them to the server
SonarQube Server	Receives analysis requests and coordinates processing
Compute Engine	Analyzes code and calculates quality metrics
Elasticsearch	Provides fast search and indexing for analysis results
Database	Stores projects, metrics, reports, users, and history
Web Server	Displays dashboards, issues, and quality reports

Integrating SonarQube with Maven & Running Analysis

Step 1: Install SonarQube

Download the Community Edition from the official SonarQube website.

After extracting it, you'll find folders like:

- sonarqube/
- ├─ bin
- ├─ conf
- ├─ data
- ├─ extensions
- ├─ logs
- └─ temp

Step 2: Start SonarQube

Windows

Go to

sonarqube/bin/windows-x86-64/

Run

StartSonar.bat

A command window opens.

Wait until you see:

SonarQube is operational

That means the server has started successfully.

Step 3: Open the Dashboard

Open your browser:

http://localhost:9000

You'll see the SonarQube login page.

Default credentials:

Username : admin

Password : admin

On the first login, SonarQube will ask you to change the default password.

Step 4: Your Maven Project

Suppose your project is:

StudentManagement

├─ src

├─ pom.xml

└─ application.properties

We need to configure pom.xml.

Step 5: Add SonarQube Maven Plugin

Inside <build>:

<build>

<plugins>

<plugin>

<groupId>org.sonarsource.scanner.maven</groupId>

<artifactId>sonar-maven-plugin</artifactId>

<version>3.11.0.3922</version>

</plugin>


```
</plugins>
```

```
</build>
```

This plugin allows Maven to communicate with SonarQube.

Step 6: Configure Sonar Properties

Inside <properties>:

```
<properties>
```

```
<sonar.host.url>
```

```
http://localhost:9000
```

```
</sonar.host.url>
```

```
</properties>
```

This tells Maven where the SonarQube server is running.

Step 7: Generate a Project Token

Modern SonarQube doesn't allow using your password for analysis.

Instead, you create a **token**.

Go to:

Profile



My Account



Security



Generate Token

Example:

Token Name

StudentProject

Click

Generate

You'll get something like:

```
sqp_8JHG4LKJHG98JKHG98JHJKHG98J
```

Copy it immediately. It won't be shown again.

Step 8: Run Sonar Analysis

Open the terminal in your project.

Run:

```
mvn clean verify sonar:sonar \
```

-Dsonar.token=YOUR_TOKEN

Example:

mvn clean verify sonar:sonar \

-Dsonar.token=sqp_abcd123456789

What happens?

1. clean removes old build files.
2. verify compiles and runs tests.
3. sonar:sonar uploads the analysis to SonarQube.

Step 9: Maven Output

You'll see logs like:

[INFO] Scanning project...

[INFO] Sensor JavaSensor

[INFO] Sensor XML Sensor

[INFO] Sensor Java CPD

[INFO] Upload report

[INFO] Analysis successful

[INFO] EXECUTION SUCCESS

This means the scan completed successfully.

Step 10: View the Dashboard

Refresh

<http://localhost:9000>

Now you'll see your project.

Example:

Project

StudentManagement

Click it.

You'll see:

Bugs

Code Smells

Coverage

Duplications

Complexity

Security

Quality Gate

What if the Server is Down?

Suppose SonarQube isn't running.

You execute:

```
mvn sonar:sonar
```

You'll get:

Connection refused

Unable to reach

http://localhost:9000

Solution:

Start SonarQube first.

What if the Token is Wrong?

You'll see:

Unauthorized

Not authorized

Authentication failed

Solution:

Generate a new token and use it.

What if Maven Isn't Installed?

Command:

```
mvn clean verify sonar:sonar
```

Output:

'mvn' is not recognized

Solution:

- Install Maven.
- Add Maven's bin folder to the system PATH.
- Verify with:

```
mvn -version
```

What if Java Version Doesn't Match?

Example:

Unsupported class version

Solution:

Use the correct JDK version (for example, Java 17 if your project targets Javas 17).

Example Code:

StudentManagementApplication.java ×

```
1 package Studentmanagement;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class StudentManagementApplication {
8
9     public static void main(String[] args) {
10         SpringApplication.run(StudentManagementApplication.class, args);
11     }
12 }
```

StudentController.java ×

```
1 package Studentmanagement.controller;
2
3 import Studentmanagement.entity.Student;
4 import Studentmanagement.repository.StudentRepository;
5 import org.springframework.web.bind.annotation.*;
6
7 import java.util.List;
8
9 @RestController
10 @RequestMapping("/students")
11 public class StudentController {
12
13     private final StudentRepository repository; 3 usages
14
15     public StudentController(StudentRepository repository) {
16         this.repository = repository;
17     }
18
19     @PostMapping
20     public Student save(@RequestBody Student student){
21
22         return repository.save(student);
23     }
24
25
26     @GetMapping
27     public List<Student> getAll(){
28
29         return repository.findAll();
30     }
31 }
32 }
```

```
StudentRepository.java ×
1 package Studentmanagement.repository;
2
3 import Studentmanagement.entity.Student;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 public interface StudentRepository 3 usages
7     extends JpaRepository<Student,Integer> {
8 }
```

```
Student.java ×
1 package Studentmanagement.entity;
2 import jakarta.persistence.*;
3 import java.util.List;
4 @Entity 6 usages
5 public class Student {
6     @Id
7     @GeneratedValue(strategy = GenerationType.IDENTITY)
8     private Integer id;
9     private String name; no usages
10    @ManyToMany(cascade = CascadeType.ALL) no usages
11
12    @JoinTable(
13        name = "student_course",
14        joinColumns = @JoinColumn(name="student_id"),
15        inverseJoinColumns = @JoinColumn(name="course_id")
16    )
17    private List<Course> courses;
18
19    public Student() {
20    }
21 }
22 }
```

Pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
```

```
<version>3.3.2</version>
<relativePath/>
</parent>

<groupId>org.example</groupId>
<artifactId>SpringMvcDemo</artifactId>
<version>1.0</version>

<properties>
  <java.version>17</java.version>
  <!-- SonarQube -->
  <sonar.host.url>http://localhost:9000</sonar.host.url>
  <sonar.projectKey>StudentManagement</sonar.projectKey>
  <sonar.projectName>StudentManagement</sonar.projectName>

  <!-- Replace with your NEW token -->
  <sonar.token>sqp_dca76f5a4f77858a7d2a7446c230f94a917408c6</sonar.token>
</properties>
<dependencies>
  <!-- Spring Boot -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
  </dependency>
  <!-- Web -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- JPA -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <!-- MySQL -->
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
  </dependency>
</dependencies>

<build>
  <plugins>

    <!-- Spring Boot -->
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

```
<configuration>
  <mainClass>Studentmanagement.StudentManagementApplication</mainClass>
</configuration>
```

```
</plugin>
```

```
<!-- SonarQube -->
```

```
<plugin>
  <groupId>org.sonarsource.scanner.maven</groupId>
  <artifactId>sonar-maven-plugin</artifactId>
  <version>3.11.0.3922</version>
</plugin>
```

```
</plugins>
```

```
</build>
```

```
</project>
```



SonarQube is starting



```
C:\WINDOWS\system32\cmd. x + v - □ ×

Starting SonarQube...
2026.07.22 14:22:35 INFO app[][o.s.a.AppFileSystem] Cleaning or creating temp directory E:\sonarqube-26.7.0.124771\temp
2026.07.22 14:22:35 INFO app[][o.s.a.es.EsSettings] Elasticsearch listening on [HTTP: 127.0.0.1:9001, TCP: 127.0.0.1:{}]
2026.07.22 14:22:35 INFO app[][o.s.a.ProcessLauncherImpl] Launch process[ELASTICSEARCH] from [E:\sonarqube-26.7.0.124771\elasticsearch]: C:\Program Files\Java\bin\java -Xms4m -Xmx64m -XX:+UseSerialGC -Dcli.name=server -Dcli.script=./bin/elasticsearch -Dcli.libs=lib/tools/server-cli -Des.path.home=E:\sonarqube-26.7.0.124771\elasticsearch -Des.path.conf=E:\sonarqube-26.7.0.124771\data\es8\config -Des.distribution.type=tar -cp E:\sonarqube-26.7.0.124771\elasticsearch\lib\*;E:\sonarqube-26.7.0.124771\elasticsearch\lib\cli-launcher\* org.elasticsearch.launcher.CliToolLauncher
2026.07.22 14:22:35 INFO app[][o.s.a.SchedulerImpl] Waiting for Elasticsearch to be up and running
Jul 22, 2026 2:22:40 PM sun.util.locale.provider.LocaleProviderAdapter <clinit>
```

⚠ Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. [Learn more](#)



ProjectsIssuesRulesQuality profilesQuality gatesAdministrationMore

StudentManagement

Project

Project onboarding

Policies

Quality profiles

Quality gate

Project

Project information

Project settings

StudentManagement

Analysis Method

Locally

Analyze your project

We initialized your project on SonarQube Community Build, now it's up to you to launch analyses!

1 Provide a token

Analyze "StudentManagement": sqp_dca76f5a4f77858a7d2a7446c230f94a917408c6

2 Run analysis on your project

What option best describes your project?

Maven

Gradle

JS/TS & Web

.NET

Python

Other (for Go, PHP, ...)

Execute the Scanner for Maven

Running a SonarQube analysis with Maven is straightforward. You just need to run the following command in your project's folder.

```
mvn clean verify org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \
-Dsonar.projectKey=StudentManagement \
-Dsonar.projectName='StudentManagement' \
-Dsonar.host.url=http://localhost:9000 \
-Dsonar.token=sqp_dca76f5a4f77858a7d2a7446c230f94a917408c6
```

Copy

Analysis

⚠ Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. [Learn more](#)

• The way in which security, reliability, and maintainability counts and ratings are calculated has changed. [Learn more in SonarQube documentation](#)



ProjectsIssuesRulesQuality profilesQuality gatesAdministrationMore

My Favorites

All

Filters

Quality Gate

Security

Reliability

Maintainability

Search projects

Perspective

Overall Status

Sort by

Name

1 project(s)

StudentManagement

Public

Last analysis: 3 minutes ago

387 Lines of Code

Java, XML

E 4

A 0

A 28

A —

0.0%

0.0%

Security

Reliability

Maintainability

Hotspots Reviewed

Coverage

Duplications

1 of 1 shown

SonarQube™ technology is powered by SonarSource Sàrl

Community Build • v26.7.0.124771 • MQR MODE

LGPL v3CommunityDocumentationPluginsWeb API

Log	Meaning
Scanning project	Sonar started reading the project
JavaSensor	Java files are being analyzed
XML Sensor	XML files are being analyzed
Upload report	Results are sent to the server
Analysis SUCCESS	Scan completed successfully